```
  def matches?(account)
    @account = account
    values_match?(@amount, account_balance)
  end

  def description
    if @as_of_date
      "have a balance of #{description_of(@amount)} as of #{@as_of_date}"
    else
      "have a balance of #{description_of(@amount)}"
    end
  end

  def failure_message
    "expected #{@account.inspect} to #{description}" + failure_reason
  end

  def failure_message_when_negated
    "expected #{@account.inspect} not to #{description}" + failure_reason
  end

private

  def failure_reason
    ", but had a balance of #{account_balance}"
  end

  def account_balance
    if @as_of_date
      @account.balance_as_of(@as_of_date)
    else
      @account.current_balance
    end
  end
end
```

Let's walk through this class from top to bottom. First, we include the RSpec::Matchers::Composable mixin on line 2.[6] This module defines a few methods for you that make composition possible, including and, or, and the === operator. It also provides helpers for you to call when you're defining a higher-order matcher, such as values_match? and description_of.

In the initializer on line 4, we store the desired amount in an instance variable so that we have something to compare against in the matches? method.

On line 8, the as_of method provides the fluent interface that lets callers write expect(...).to have_a_balance_of(...).as_of(...). It looks a lot like the DSL version, except that here we have to return self explicitly. Without that line, as_of would return

6. http://rspec.info/documentation/3.6/rspec-expectations/RSpec/Matchers/Composable.html